



CRF Application Portal

Maggie Kuffskie, Mara Schimmel, and Olivia Christie

Table of Contents

1. Planning.....	2
1.1 Project Overview.....	2
1.2 Motivation.....	2
1.3 Primary Objectives.....	3
1.4 Scope.....	3
1.5 Technical Skill Requirements.....	4
1.6 Contribution Chart.....	4
2. Analysis.....	5
2.1 Elicitation Strategies.....	5
2.2 Formal Requirement Statement.....	5
2.3 Master Test Plan.....	6
3. Design.....	7
3.1 Logical Design.....	7
3.2 Physical Design.....	11
3.3 Security Considerations.....	11
4. Implementation.....	12
4.1 Code Highlights.....	12
4.2 Record of Testing.....	13
5. Review.....	15
5.1 Problems Encountered.....	15
5.2 Proudest Moments.....	16
5.3 Lessons Learned.....	16
5.4 Conclusions.....	17
Appendix A - Master Test Plan.....	17
Appendix B - Wireframes.....	25

1. Planning

1.1 Project Overview

This project will consist of a website designed for Rollins College's Collaborative Research Fellowship (CRF). The website will consist of five main pages:

1. A homepage with a general overview of the CRF program
2. An FAQ page which will provide answers to frequently asked questions about the program
3. A page of past proposals to provide examples
4. A resource page with commonly used libraries and sources for various departments' proposals
5. A portal where users can upload or review proposals for the program, depending on their profile type

The main emphases of this project will be (1) easy access to information and (2) the secure storage of user data. The FAQ page will reduce the number of questions directed to the program's head, and will provide easy access to information through the use of a simple AI chatbot that can answer questions pertaining to the FAQ's contents. The resource page will help users in the process of developing a proposal by providing sources commonly used in previous years. The past examples page will help define students' expectations of what research they can accomplish in the CRF program based on projects from former participants.

The most significant aspect of this project is the proposal portal. In this portal, users will have one of three types of accounts: student, faculty, or administrator. Student accounts will be able to upload proposals, cover letters, and a timeline of the proposed research. Faculty accounts will be able to access all proposals where they are linked as the research advisor, as well as upload the associated budgets and research dates for each proposal. Finally, administrators will be able to access all information pertaining to all proposals, and will also have the ability to accept, deny, or request additional information from a specific project.

1.2 Motivation

In the ideal form of the existing system, students and faculty create a proposal and submit it to the CRF coordinator for review. The coordinator then reviews all of the proposals with a board of administrators and emails the applicants to inform them of their proposal's acceptance or denial. In practice, this results in varying standards for proposals across different departments, causing confusion for applicants, and strain on the program coordinator to organize all of the submissions' review.

Additionally, the lack of quick access to information about the CRF program results in any questions being directed to the program coordinator. This once again places undue stress on

the coordinator to handle all applicants' questions, which often relate to time sensitive aspects such as deadlines. The result is a longer wait time for information, delaying both the applicants and the administrators.

The centralization of the proposal admission system will therefore benefit both those submitting proposals as well as the administrators who have to review them. Applicants will have easy access to information, reducing confusion and by extension reducing the need for the program coordinator to answer dozens of questions. The centralization will also help to standardize the submission process, which will help applicants in identifying the requirements and administrators in their review. The result of this project will be a more streamlined experience for all involved in the CRF program, producing a more efficient system that will help to develop research in the Rollins community.

1.3 Primary Objectives

The primary objectives of this project are thus:

1. Provide easy and quick access to information needed to create a CRF research proposal
2. Provide a central portal where CRF candidates can submit all required materials
3. Create a single, secure platform for the access of all proposal-related materials

1.4 Scope

This project will streamline the process of proposal submission and review for all those involved in the CRF program. It will accomplish this by providing a portal for CRF candidates and research advisors to submit all required information and for administrators to view and appraise proposals. The project will additionally provide information that will help in the creation of CRF proposals.

The users of this system will be constrained to Rollins undergraduate students and Rollins College faculty. All aspects of the project will be completed with minimal spending required, ideally being accomplished fully through free or open-source resources. Additionally, the project will be fully operational by April 20th, 2025, with a working prototype completed by March 17th, 2025. Finally, the project will be primarily self-contained, relying on external sources as little as possible in order to make integration into Rollins systems easier.

It is important to note that this system is intended to help students at either the beginning or end of the application process. This project will not provide any tools for editing proposals, nor is it intended to be used once a research project has begun. Additionally, this system is not intended for any applications other than the Rollins College CRF program. This project will also not involve any of the budgetary or schedule calculations needed in the administration's review process. The administrators' scope will be limited to only the viewing and approval/denial of completed submissions.

1.5 Technical Skill Requirements

The technical skills required in this project were centered around web development, server storage, and AI development. Included in the table below are some programs which did not end up being used in the final product but still influenced its development, such as Dropbox and Supabase.

Technical Skill	Maggie Kuffskie	Mara Schimmel	Olivia Christie
HTML	✓	✓	✓
CSS	✓	✓	✓
JavaScript	✓	✓	✓
Firebase	□	✓	□
AI Development	□	□	□
Azure	✓	□	□
Supabase	□	✓	□
Dropbox	✓	□	✓
Node.js	□	✓	□

1.6 Contribution Chart

The contributions for this project are shown below.

Maggie Kuffskie	Mara Schimmel	Olivia Christie
Non-Portal HTML	Portal HTML	Resource Page Content
All Styling	Interviews	Interview Scheduling
Home Page Content	Interview Write Ups	FAQ Content
Interviews	Sign-up Javascript	All Bilbot Art
Interview Write Ups	Application Javascript	Presentations
Video Interview Editing	Node.js Experimentation	Testing and Validation
FAQ Content	Server Discovery	
Chatbot Implementation	Server Implementation	
Presentations	Testing and Validation	
UML Diagrams	UML Diagrams	
Documentation		

2. Analysis

2.1 Elicitation Strategies

The main elicitation strategies for this project's requirements were observation and interviews. Since this project's development happened in tandem with the CRF program's application window for the Summer of 2025, we were able to observe the application process from near start to finish and make notes of any common issues that users experienced. Interviews were conducted with the current head of the CRF program, Dr. Christopher Fuse, as well as with several students who participated in Rollins' undergraduate research program.

2.2 Formal Requirement Statement

Functional Requirements	Non-Functional Requirements
Allow users to specify the account type (student, faculty, or administrator)	Follow visual formatting consistent with official Rollins pages
Verify the log-in information before allowing access to an account according to server information	Be visually appealing on desktop devices
Allow student users to upload proposals and project timelines	Only accept and return pdf file type submissions
Allow faculty users to upload project budgets and dates	Provide 24 hour notice if the system will be unavailable for more than 10 minutes
Allow student and faculty users to upload their name, R Number, and cover letter	Store proposal information for at least 1 year after the CRF deadline
Allow students to link their account to their associated faculty	Be fully functional on commonly used browsers
Allow administrators to search for a specific project by student or faculty	Be primarily developed in HTML, CSS, and Javascript
Allow administrators to approve or deny an application	Be intuitive and easy to use
Allow administrators to approve new administrator accounts	
Allow administrator users access to all submitted projects	
Display the status of a proposal when viewed as either not submitted, under review, approved, or denied	
Prohibit changes to an application's contents once submitted for review	
Provide answers to commonly asked questions	

Functional Requirements	Non-Functional Requirements
Utilize an AI API to answer questions relating to the FAQ	
Provide examples of commonly used resources	
Allow users to search for a specific resource by department	
Provide an overview of the CRF program	
Provide contact information for CRF administrators	
Divide department-specific resources into sections based on the department	
Provide links to resources such as library sources, websites, journals, etc.	
Limit student users' access to only their project	
Limit faculty user's access to only projects they are confirmed to be advisors on	
Showcase past project examples in the form of video or written testimonials	
Automatically log out when browser window closes	

2.3 Master Test Plan

The Master Test Plan (available in Appendix A) was developed with six main categories of tests. These categories consist of:

1. Sign Up
2. Log In / Log Out
3. Data Access and Display
4. Data Upload
5. Proposal Submission
6. Proposal Review

The tests were divided into these categories due to the different restrictions and security measures needed for each case.

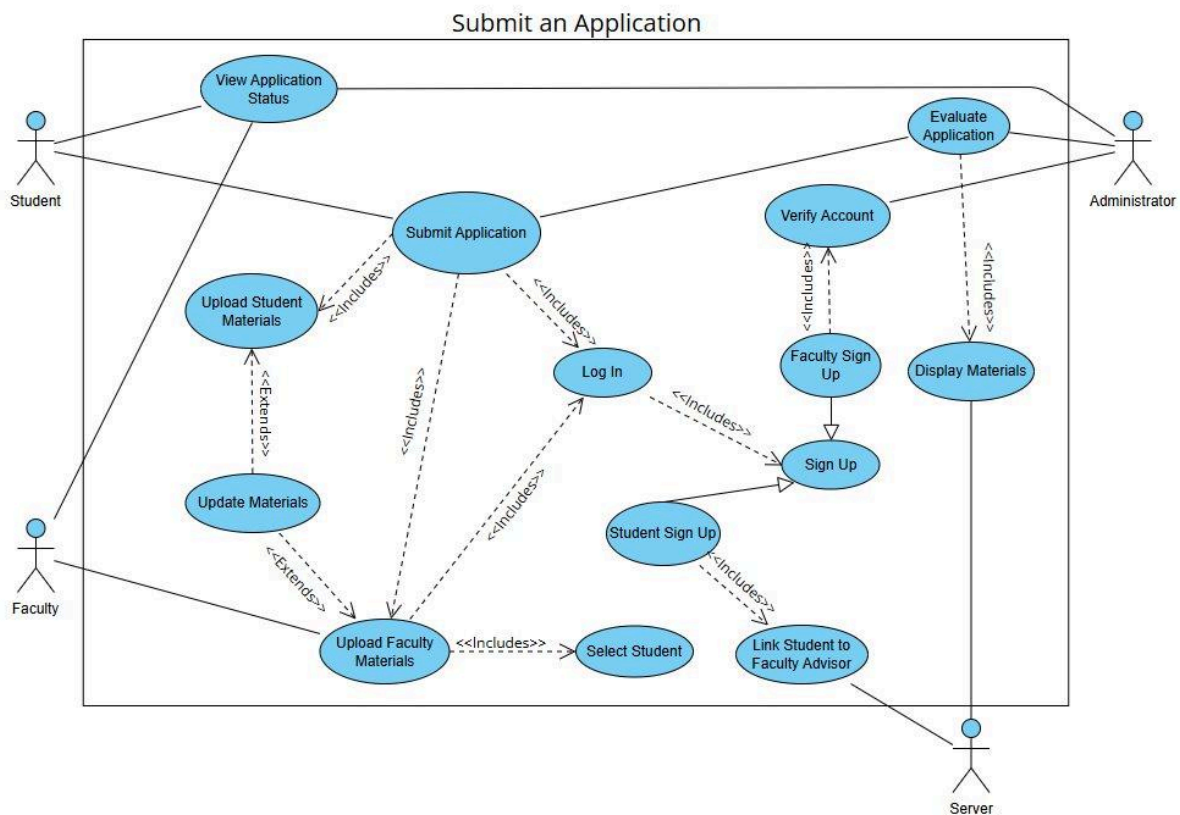
Tests focused on signing up and logging in or out primarily focused on verifying data specific to an individual account. These tests were vital to ensure that no one could gain immediate access to someone else's data, and to ensure that no one could gain access to account with higher security privileges. Tests focused on data access and display again emphasized this need for security, but instead accomplished this by verifying adequate separation of data within the server. Finally, tests relating to proposal submission and review focused on limiting access to these higher clearance functions, and on verifying that an account's status would be accurate and up to date.

3. Design

3.1 Logical Design

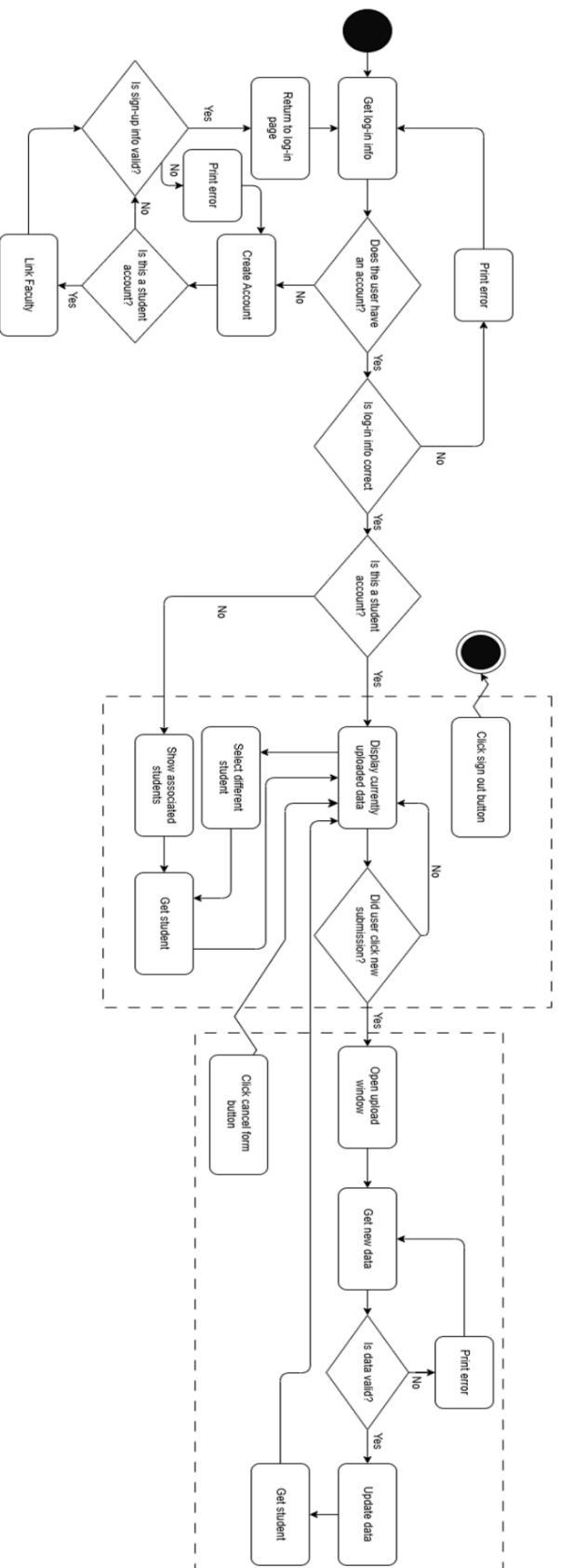
The logical design of this project began with wireframes. Each page of the website was illustrated in order to develop the layout and user experience of the system. In the development of the wireframes, consistency with current Rollins College formatting was among the highest priorities in order to ensure easy integration with current Rollins pages. These wireframes are available in Appendix B.

Following the solidification of the website design, Use Case Diagrams and Activity Diagrams were created to define the front and back end of the system respectively. The Use Case diagram describes the different users' experience of the application system as either a student, faculty, or administrator, as well as the actions of the server.

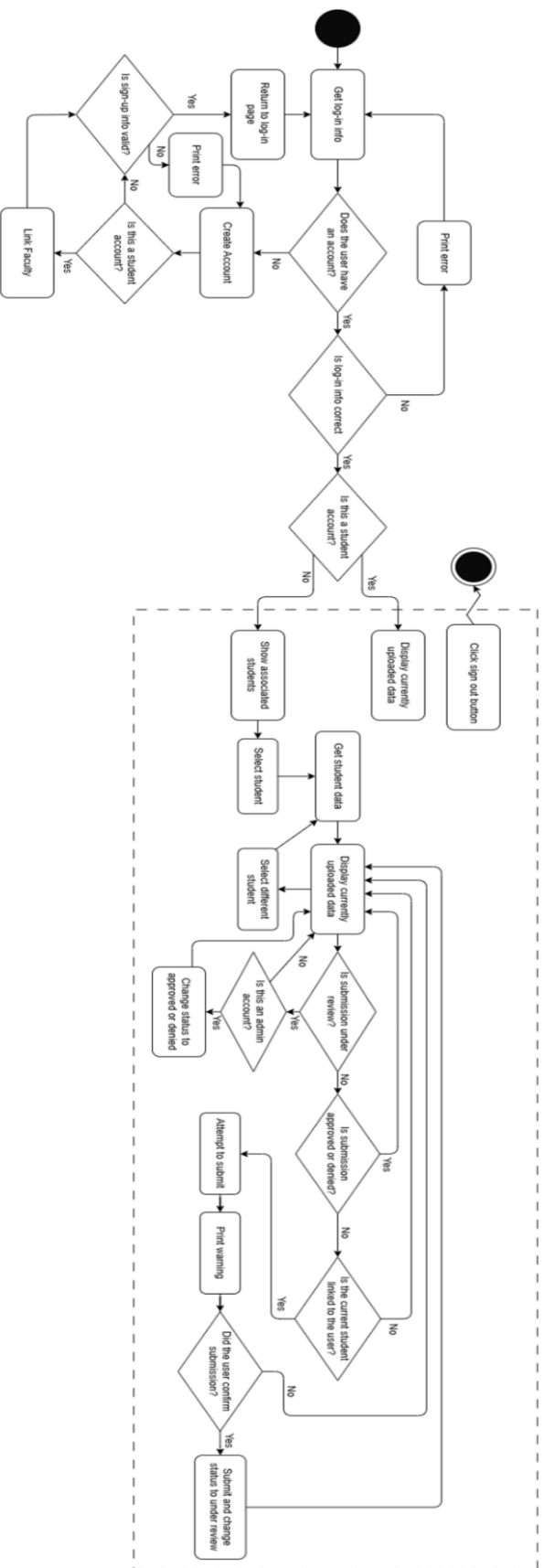


The Activity Diagrams focused on two main processes in the backend: application creation and application submission. These two aspects of the system were divided due to the fact that only faculty and administrator accounts may submit an application.

Create an Application

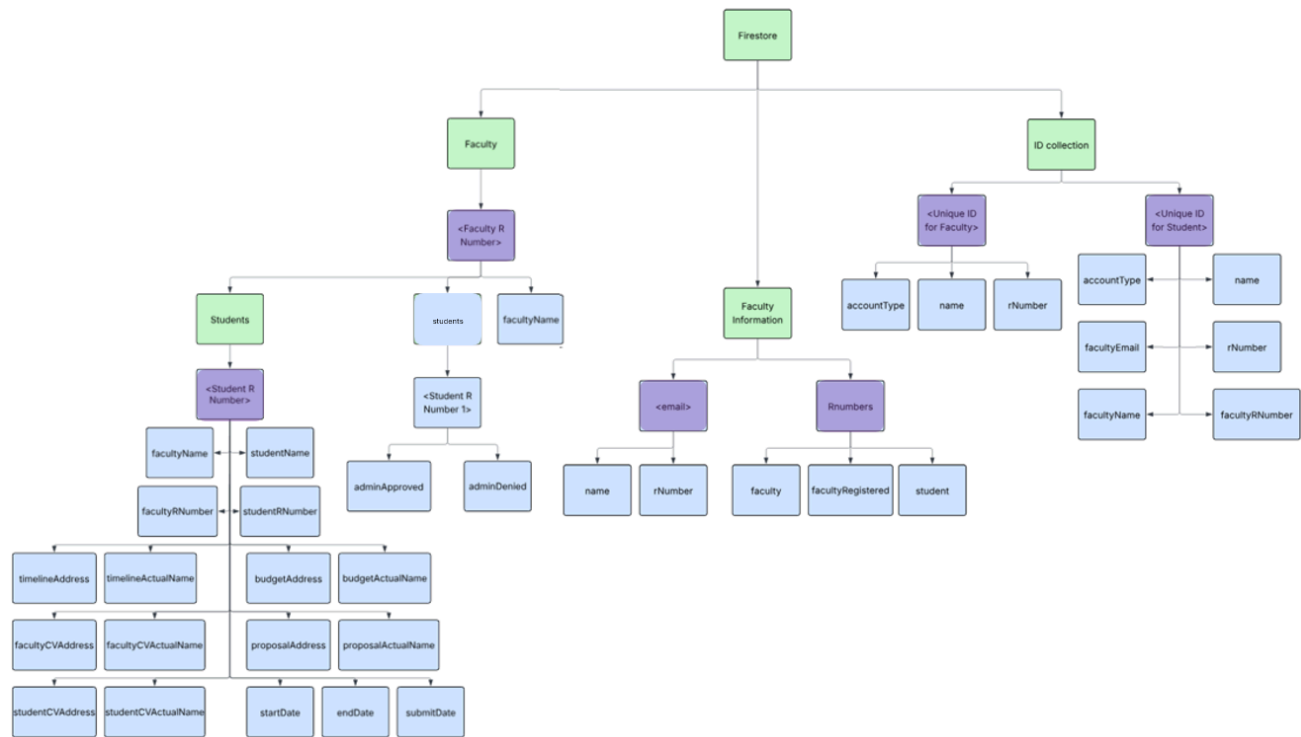


Submit an Application



3.2 Physical Design

Firestore Structure



The physical design for this system consisted primarily of solidifying the database layout. Sections in Firebase were designated as either collections (green), documents (purple), or data (blue). These designations allowed us to determine the overall hierarchy of how data would be stored in the server, as well as how individual data points needed to be classified.

3.3 Security Considerations

The security considerations for this project focused primarily on ensuring data security. As this portal would contain all application data for the program, including aspects such as budgetary information, it was important that student- and faculty-submitted data could only be accessed by the intended parties.

This required security was ensured with a number of methods. First, all passwords are required to have a minimum length of 8 characters and to contain at least one uppercase, lowercase, numeric, and non-alphanumeric character. This was implemented to help prevent brute force attacks from gaining access to an account.

Second, a faculty account can only be created if a user signs up with a pre-approved R number and email pair. These pairs are stored in the system's database and can be updated as different faculty become eligible or ineligible to participate in the CRF program. This restriction

ensures that faculty accounts, which have a higher level of access in the system, are only able to be created by parties that know both an eligible faculty's email and their R number.

Finally, all data was stored in an external server, independent of the system that provides the website and portal. This extra layer of separation helps reduce the chance of an attacker reading private server information without first being verified through log-in credentials.

4. Implementation

4.1 Code Highlights

One portion of this project's development that took the longest to implement was the display of a user's submitted data. This process involved a large amount of work to create a seamless user experience while still ensuring quick and reliable data retrieval from the server. Shown below is a segment of code that helps to execute the data retrieval.

```
267 let studentForm = document.getElementById("form-student")
268 studentForm.addEventListener('submit', studentUpload)
269
270 if (studentData !== undefined) {
271   // Populate the submitted student data
272
273   let students = studentSubmissionData['students']
274
275   document.getElementById("status-student").innerHTML = students[studentRNumber]['status']
276   document.getElementById("submitted-student").value = studentData['submitDate']
277
278   document.getElementById("student-name-student-display").parentNode.classList.remove("hidden")
279   document.getElementById("student-name-student-display").value = studentData['studentName']
280   document.getElementById("student-rNumber-student-display").value = studentData['studentRNumber']
281
282   displayFileData = {
283     'timeline': {'address': studentData['timelineAddress'], 'name': studentData['timelineActualName']},
284     'proposal': {'address': studentData['proposalAddress'], 'name': studentData['proposalActualName']},
285     'studentCV': {'address': studentData['studentCVAddress'], 'name': studentData['studentCVActualName']}
286   }
287
288   document.getElementById("timeline-student-display").addEventListener("click", downloadTimeline)
289   document.getElementById("proposal-student-display").addEventListener("click", downloadProposal)
290   document.getElementById("student-cv-student-display").addEventListener("click", downloadStudentCV)
291
292   if (students[studentRNumber]['status'] === "In Progress") {
293     // Allow the user to upload data
294     document.getElementById("update-student-info").addEventListener("click", populateStudentForm)
295     document.getElementById("update-student-info").classList.remove("hidden")
296   } else {
297     document.getElementById("update-student-info").removeEventListener("click", populateStudentForm)
298     document.getElementById("update-student-info").classList.add("hidden")
299   }
300
301   document.getElementById("prev-submission-student").classList.remove("hidden")
302 }
```

This code section specifically shows the population of a student's previously submitted data, including their name, R number, any submitted documents, and submission status. This section of code is also what allows the user to download previously submitted documents, as seen in lines 288 through 290. The population of faculty and administrator data follows an

extremely similar format, meaning that the development of this code section was integral to the functionality of the application portal.

Another code section worth highlighting is the section that allows for new user sign ups. In this section of code, shown below, 5 separate functions were created to verify that a new user's sign-up data was valid and available to use. The functions shown in this code section allow for the system to identify if any sign up data has already been used by a previous user. Additionally, this section of code enforces the password requirements described in the Security Considerations section, helping to keep user applications secure.

```
63  async function main() {
64      const createAccountForm = document.getElementById("sign-up")
65      const accountTypeSelect = document.getElementById("account-type")
66
67      createAccountForm.addEventListener("submit", createAccount)
68      accountTypeSelect.addEventListener("change", toggleStudentView)
69
70      document.getElementById("email").addEventListener("blur", verifyUserEmail)
71      document.getElementById("password").addEventListener("input", passwordListener)
72
73      document.getElementById("rnumber").addEventListener("blur", confirmRNumbers)
74      document.getElementById("confirm-rnumber").addEventListener("blur", confirmRNumbers)
75
76      if (auth.currentUser) {
77          console.log(auth.currentUser)
78      } else {
79          console.log("Not signed in")
80      }
81  }
```

4.2 Record of Testing

Each of the tests listed in the Master Test Plan (available in Appendix A) were conducted by members of the development team. The results of all test cases matched the expected results. Therefore, all tests have passed.

Sign-Up				
1.01	1.02	1.03	1.04	1.05
1.06	1.07	1.08	1.09	1.10
1.11	1.12	1.13	1.14	1.15

Log-In / Log-out				
2.01	2.02	2.03	2.04	2.05
2.06	2.07	2.08	2.09	2.10
2.11	2.12	2.13	2.14	2.15
2.16	2.17			

Data Display				
3.01	3.02	3.03	3.04	3.05
3.06	3.07	3.08	3.09	3.10
3.11	3.12	3.13	3.14	3.15
3.16	3.17	3.18	3.19	3.20
3.21	3.22	3.23		

Upload Data				
4.01	4.02	4.03	4.04	4.05
4.06	4.07	4.08	4.09	

Submission				
5.01	5.02	5.03	5.04	5.05
5.06	5.07	5.08		

Decision				
6.01	6.02	6.03		

In addition to completing the above test cases, the system was also shown to potential users to determine whether user acceptance was reached. The system was first shown to 3 past CRF participants, who were asked to use the system to submit an application and compare the experience to their previous submission process. While this sample size is admittedly small, it does represent 10% of the typical CRF admission size of approximately 30 people. In all three cases, the users stated that the portal's experience was more streamlined and more convenient than the existing system. The system was also shown to Dr. Chris Fuse, the current head of the CRF program. His acceptance was confirmed for both the information provided and the portal functionality.

5. Review

5.1 Problems Encountered

Throughout the creation of this project, three main problems were encountered. The first of these arose during the implementation of Bilbot Baggins, the AI Chatbot that helps answer questions from the FAQ page. This chatbot was implemented using Microsoft Azure, which none of the team had experience with before starting this project. This learning curve in itself was a challenge, but there was also the problem of pricing.

Microsoft Azure offers a student account which temporarily supplies the account with credits, but that credit is limited. Since the team was inexperienced with Microsoft Azure, we underestimated how much it cost to test the chatbot. This led to the Azure student account running out of funds near the end of the project, meaning the chatbot had to be recreated on a new student account.

Another problem encountered was the storing of user files. The initial plan was to implement Supabase, an open-source database infrastructure, to hold files associated with applications. While working with Supabase however, it was decided that the structure of the system was not cohesive with how we wanted to organize the backend data hierarchy. The backend was therefore switched to Firebase, another service that offers server simulation. This unfortunately meant that the project had to change its backend mid-way through development, resulting in significant changes to the javascript.

The final problem that was faced during this project's development occurred when trying to gather testimonials regarding previous participants' research. The number of CRF researchers per year is small, typically around 20-30 people, meaning that the total pool of possible interviewees was much smaller than hoped. Additionally, scheduling conflicts meant that students often had to reschedule, resulting in delays in interviews. This was mitigated slightly by some people offering to provide written testimonials instead, reducing the scheduling burden.

5.2 Proudest Moments

The proudest moments during this project can be summarized into three main topics. The first of these was the creation of a way for users to upload and download files. Explained briefly in the Code Highlights section, the ability for users to upload and download files involved both the implementation of an external server and the creation of numerous functions. The finalization of these functions meant that users could easily submit their applications while being sure that their data would be stored securely.

The second moment of pride came from the completion of the website's stylization. A key point throughout the project's development was that the project should be cohesive with Rollins College's existing web design. This involved balancing the typical Rollins colors and formats with an attempt to create a new, interesting user experience. Completed around early April, the completion of styling allowed us to see the final product as users will.

The final highlight of this experience was the creation of the AI chatbot Bilbot Baggins and his mascot. The implementation of Bilbot Baggins proved a challenge, but also provided great satisfaction seeing the AI provide quick and efficient answers to commonly asked questions. The creation of the mascot also helped team morale, and became an integral part of the website.



5.3 Lessons Learned

Throughout the development of the CRF website and application portal, we as a team learned the value of long and short term planning. This was all of our first experiences with the Scrum Model of development, and we found that the large-scale planning allowed us to better prioritize and schedule aspects that would take several weeks to complete, such as interviews and server implementation. On the short term side, we learned the value of making adjustments as needed, like we did during the problem with Azure pricing. Overall, we as a team learned both how to prioritize large-scale tasks while also leaving room for unexpected problems or delays.

We as a team also learned valuable lessons regarding client collaboration. This project was developed with Dr. C Fuse, the current head of the CRF program. This collaboration allowed us to learn how to take requirements from a user and transform them into a cohesive system. We also gained experience in working within a framework, as this project was developed to be easily integrated into Rollins websites.

Finally, though it's not a typical lesson, we learned that small details can make both users and developers have a better experience. The Bilbot Baggins mascot was not originally part of the plan, but he became a constant presence in development following his creation. The creativity and small bit of whimsy involved in his design allowed us to make the website more welcoming to users while still maintaining the needed clarity. Additionally, the presence of a cute mascot helped keep developer morale up through all the difficult moments.

5.4 Conclusions

We as a team are incredibly proud of this project. We feel that we created a project that not only meets expectations, but exceeds them. The created project will allow an easier and more centralized experience to all participants in the CRF program, from students merely interested in the program all the way up to those who run it. The project can easily be implemented into Rollins College servers and websites, providing a great opportunity for Rollins College to increase its students' interest in and access to research.

We additionally learned quite a lot through this project. Each of us gained new technical skills which we will likely continue to use in our futures. Additionally, due to the client collaboration and interviewing aspects of this project, we learned how to balance technical requirements with human connections and interactions. Above all else, however, we learned the value of collaboration with those in and outside of our discipline, and the importance of making things fun.

Appendix A - Master Test Plan

Test ID	General Case	Specific Details	Success Criteria
Sign up			
1.01	Valid sign-up	All data is valid for a new user	User should be logged in and taken to their respective starting page
1.02	Valid sign-up	Non-rollins email	Error message should appear once user clicks out of email input
1.03	Valid sign-up	Email already in use	Error message should appear when user tries to sign up
1.04	Valid sign-up	Not an email	Error message should appear once user clicks out of email input
1.05	Valid sign-up	Invalid R Number	Error message should appear once user clicks out of R Number input
1.06	Valid sign-up	R Number already in use	Error message should appear when user tries to sign up
1.07	Valid sign-up	Password shorter than 8 characters	Sign up should be prohibited
1.08	Valid sign-up	Password does not contain lowercase character	Sign up should be prohibited
1.09	Valid sign-up	Password does not contain uppercase character	Sign up should be prohibited
1.10	Valid sign-up	Password does not contain a number	Sign up should be prohibited
1.11	Valid sign-up	Password does not contain a non-alphanumeric character	Sign up should be prohibited
1.12	Valid sign-up	Faculty email used for student account	Error message should appear to prohibit email
1.13	Valid sign-up	Faculty email without corresponding R Number	Error message should appear notifying of incorrect R number
1.14	Link faculty as student	Faculty already signed up	Sign up should proceed as normal
1.15	Link faculty as student	Faculty has not signed up	Sign up should proceed as normal AND update once faculty signs up

Test ID	General Case	Specific Details	Success Criteria
Log-in			
2.01	Valid log-in (student)		User should see the student starting page with respective data
2.02	Valid log-in (faculty)		User should see the faculty starting page with respective data
2.03	Valid log-in (admin)		User should see the admin starting page with respective data
2.04	Invalid log-in (student, incorrect password)		Login screen should show an incorrect login message
2.05	Invalid log-in (faculty, incorrect password)		Login screen should show an incorrect login message
2.06	Invalid log-in (admin, incorrect password)		Login screen should show an incorrect login message
2.07	Invalid log-in (non-existing email, existing password)		Login screen should show an incorrect login message
2.08	Student Log-out	From Data Display	The page should close any displayed data and statuses. It should then return to the login screen
2.09	Student Log-out	From Uploading Data Screen	The user should not be able to sign out while the upload box is open
2.10	Faculty Log-out	From Data Display, No Student Selected	The page should return to the login screen
2.11	Faculty Log-out	From Display Data, Student Selected	The page should close all displayed data and statuses, then then return to the login screen
2.12	Faculty Log-out	From Uploading Data Screen	The user should not be able to sign out while the upload box is open
2.13	Faculty Log-out	From Submitting Proposal Screen	The user should not be able to sign out while the submit for review box is open
2.14	Admin Log-out	From Display Data, No Student Selected	The page should return to the login screen
2.15	Admin Log-out	From Display Data, Student Selected	The page should close all displayed data and statuses, then return to the login screen
2.16	Admin Log-out	From Uploading Data Screen	The user should not be able to sign out while the upload box is open
2.17	Admin Log-out	From Submitting Proposal Screen	The user should not be able to sign out while the submit for review box is open
Data Display			

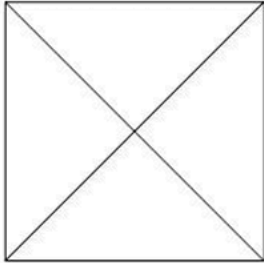
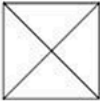
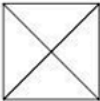
Test ID	General Case	Specific Details	Success Criteria
3.01	Display Student Data	No data	Starting page should print a no submission data found message on the screen and allow the user to submit their first data, as well as show the status as "Not Started Yet"
3.02	Display student Data	In progress	Starting page should show the data last uploaded by the student and allow the user to resubmit, as well as show the status as "In Progress"
3.03	Display student Data	Under review	Starting page should show the data last uploaded by the student and prevent further submission
3.04	Display student Data	Approved	Starting page should show the data last uploaded by the student and show "Approved" in the status and fox logo, should prevent further submission
3.05	Display student Data	Denied	Starting page should show the data last uploaded by the student as well as "Denied" in the status and fox logo, should prevent further submission
3.06	Display Faculty Data	No students	Starting page should print a no students found message and prevent opening any student data
3.07	Display Faculty Data	No data	Starting page should not allow the user to select a student with no previous submission data
3.08	Display Faculty Data	In progress	Starting page should show the last data uploaded by the student and faculty and allow the user to submit or resubmit data. It should also show "In Progress" in the status and fox logo. If there is both student and faculty data uploaded, it should allow the user to submit it for review.
3.09	Display Faculty Data	Under review	Starting page should show the last data uploaded by the student and faculty as well as "Under Review" in the status and fox logo. It should also prevent further submission
3.10	Display Faculty Data	Approved	Starting page should show the last data uploaded by the student and faculty as well as "Approved" in the status and fox logo. It should also prevent further uploads or submissions
3.11	Display Faculty Data	Denied	Starting page should show the last data uploaded by the student and faculty as well as "Denied" in the status and fox logo. It should also prevent further uploads or submissions
3.12	Display Admin Data	No students	Starting page should show the last data uploaded by the student and faculty and allow the user to submit or resubmit data. It should also show "In Progress" in the status and fox logo

Test ID	General Case	Specific Details	Success Criteria
3.13	Display Admin Data	No data	Starting page should not allow the user to select a student with no previous submission data
3.14	Display Admin Data	In progress	Starting page should show the last data uploaded by the student and faculty and allow the user to submit or resubmit data if the student belongs to the admin. It should also show "In Progress" in the status and fox logo. If there is both student and faculty data uploaded, it should allow the user to submit it for review
3.15	Display Admin Data	Under review	Starting page should show the last data uploaded by the student and faculty as well as "Under Review" in the status and fox logo. It should also prevent further submission and allow the user to approve or deny the submission
3.16	Display Admin Data	Approved	Starting page should show the last data uploaded by the student and faculty as well as "Approved" in the status and fox logo. It should also prevent further submission, approval, denial, or submission for review
3.17	Display Admin Data	Denied	Starting page should show the last data uploaded by the student and faculty as well as "Denied" in the status and fox logo. It should also prevent further submission, approval, denial, or submission for review
3.18	Faculty Switch Students	No students	The starting page should show up as explained by Display Faculty Data when there are no students
3.19	Faculty Switch Students	1 Student	The starting page should allow the user to select their only student with previous data. Once the student is selected and the page is loaded, the user should not be able to select another student
3.20	Faculty Switch Students	2+ Students	The starting page should allow the user to select between their students with previous data. Once the student is selected and the page is loaded, the user should be able to switch to another student when not currently uploading data or submitting for review
3.21	Admin Switch Students	No students	The starting page should show up as explained by Display Admin Data when there are no students
3.22	Admin Switch Students	1 Student	The starting page should allow the user to select their only student with previous data. Once the student is selected and the page is loaded, the user should not be able to select another student

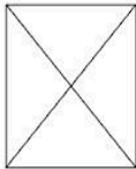
Test ID	General Case	Specific Details	Success Criteria
3.23	Admin Switch Students	2+ Students	The starting page should allow the user to select between their students with previous data. Once the student is selected and the page is loaded, the user should be able to switch to another student when not currently uploading data or submitting for review
Upload Data			
4.01	Student Upload	Valid (only pdf)	The upload box should close, and the page should reflect the data display for Display Student Data while In progress
4.02	Student Upload	Invalid	Any invalid files uploaded should be removed from the upload box and an error message should show up as an alert
4.03	Student Upload	Cancel	The upload box should close and the page should display for Display Student Data while In progress
4.04	Faculty Upload	Valid	The upload box should close, and the page should reflect the data display for Display Faculty Data while In progress
4.05	Faculty Upload	Invalid	Any invalid files uploaded should be removed from the upload box and an error message should show up as an alert. If the start or end date are invalid, an error message should show up as an alert
4.06	Faculty Upload	Cancel	The upload box should close and the page should display for Display Faculty Data while In progress
4.07	Admin Upload	Valid	The upload box should close, and the page should reflect the data display for Display Admin Data while In progress
4.08	Admin Upload	Invalid	Any invalid files uploaded should be removed from the upload box and an error message should show up as an alert. If the start or end date are invalid, an error message should show up as an alert
4.09	Admin Upload	Cancel	The upload box should close and the page should display faculty data and In Progress status
Submit			
5.01	Faculty Submit	Missing data, In Progress	The user should not be able to submit for review while there is no faculty data uploaded

Test ID	General Case	Specific Details	Success Criteria
5.02	Faculty Submit	Complete data, In Progress, Confirm	The user should be able to submit a proposal for review. Upon clicking the submit for review button, a confirmation box should appear, the confirmation box should close upon confirmation and the page should update to Display Faculty Data while Under review
5.03	Faculty Submit	Already submitted	The user should not be able to submit for review after a proposal has already been submitted for review
5.04	Faculty Submit	Complete data, In Progress, Cancel	When the user clicks the cancel button while in the confirmation box, the box should close and reflect Display Faculty Data while In Progress
5.05	Admin Submit	Missing data	The user should not be able to submit for review while there is no faculty data uploaded
5.06	Admin Submit	Complete data, In Progress	The user should be able to submit a proposal for review as long as the student in question belongs to the admin. Upon clicking the submit for review button, a confirmation box should appear, and upon clicking confirm, the confirmation box should close and the page should update to Display Admin Data while Under review
5.07	Admin Submit	Already submitted	The user should not be able to submit for review after a proposal has already been submitted for review
5.08	Admin Submit	Cancel	When the user clicks the cancel button while in the confirmation box, the box should close and reflect Display Admin Data while In Progress
Decision			
6.01	Admin change status	In progress OR Approved OR Denied	The user should not be able to approve or deny a proposal while its status is "In Progress", "Approved", or "Denied"
6.02	Admin change status	under review, to approve	When the user clicks the approve button while the proposal is "Under Review", the page should update to reflect an approved status for that student
6.03	Admin change status	under review, to deny	When the user clicks the deny button while the proposal is "Under Review", the page should update to reflect an denied status for that student

Appendix B - Wireframes

Home Title			
Resources	Portal	Examples	FAQ
Overview ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~ ~~~~~			
People			
 ~~~~~ ~: ~:			
 ~~~~~ ~: ~:			

Resources			
HOME	Portal	Examples	FAQ
A, B, C, D, E,			
Anthropology ~~~~~ ~~~~~ ~~~~~			
Art History ~~~~~ ~~~~~ ~~~~~			

Examples			
HOME	Resource	Portal	FAQ
2024  Name: _____ Project: _____ _____ _____ Name: _____ Project: _____ _____ _____ _____ 			

Frequently Asked Questions			
HOME	Resource	Examples	Portal
Question 1: _____ _____ _____ _____ Question 2: _____ _____ _____			

(Student)

Portal				Log out
HOME	Resource	Examples	FAQ	
Name: R Number: Timeline: Upload File Proposal: Upload File Cover Letter: Upload File Status: _____ Submitted: _____ Update Info				

Log-in pop up

User name
Password

(Faculty)

Portal

Log out

[HOME](#)
[Resources](#)
[Examples](#)
[FAQ](#)

Student

Name:

R Number:

Date:

MONTH

Status:

Submitted:

[Update info](#)

Budget:

Cover letter:

(Admin)

Portal

Log out

[HOME](#)
[Resources](#)
[Examples](#)
[FAQ](#)

Student

Student:

Name:

R Number:

Cover letter:

Faculty:

Name:

R Number:

Cover letter:

Date:

MONTH

Status:

Submitted:

[Approve](#)

[Deny](#)

[Request info](#)

Timeline:

Proposal:

Budget: